

RE — Exercices Heap (malloc / free / realloc / calloc)

Focus sur la mémoire dynamique. En RE, chaque `call malloc` est un indice sur les structures de données du programme. Savoir lire le heap = savoir comprendre un malware.

```
cd notes/reverse-engineering/lecon
```

Compile toujours avec : `gcc -g -O0 -o nom nom.c`

Exercice H1 — Tableau dynamique qui grandit (malloc + realloc)

Écris un programme qui :

- Commence avec un tableau de 2 entiers (`malloc`)
- Demande des nombres à l'utilisateur en boucle
- Quand le tableau est plein, double sa taille avec `realloc`
- L'utilisateur tape `-1` pour arrêter
- Affiche tous les nombres saisis

Résultat attendu :

```
Nombre (-1 pour stop): 10
Nombre (-1 pour stop): 20
Nombre (-1 pour stop): 30
[realloc: 2 -> 4]
Nombre (-1 pour stop): 40
Nombre (-1 pour stop): 50
[realloc: 4 -> 8]
Nombre (-1 pour stop): -1
Résultat: 10 20 30 40 50
```

Indice RE : `realloc` peut déplacer le bloc en mémoire. En asm, le nouveau pointeur retourné dans `rax` peut être différent de l'ancien.

► Solution

Exercice H2 — calloc vs malloc (calloc)

Écris un programme qui :

- Alloue un tableau de 10 entiers avec `malloc`
- Affiche les 10 valeurs (elles sont **non initialisées** → garbage)
- Alloue un tableau de 10 entiers avec `calloc`
- Affiche les 10 valeurs (elles sont **à zéro**)

Résultat attendu :

```
malloc   : 0 0 1234567 -42 ... (valeurs aléatoires)
calloc   : 0 0 0 0 0 0 0 0 0 0
```

Indice RE : `calloc(n, size) = malloc(n*size) + memset(0)`. En asm tu verras un `call calloc` ou un `call malloc` suivi de `rep stosb` (remplissage à zéro).

► Solution

Exercice H3 — Tableau de strings dynamique (malloc + malloc)

Écris un programme qui :

- Demande combien de noms l'utilisateur veut entrer (`n`)
- Alloue un tableau de `n` pointeurs `char*` avec `malloc`
- Pour chaque nom : alloue un buffer de 64 bytes, lit le nom dedans
- Affiche tous les noms avec leur adresse heap
- Libère **chaque buffer** puis le tableau de pointeurs

Résultat attendu :

```
Combien de noms ? 3
Nom 1: Alice
Nom 2: Bob
Nom 3: Charlie
---
[0x5555...] Alice
[0x5555...] Bob
[0x5555...] Charlie
```

Indice RE : un `char **` = tableau de pointeurs vers des strings. En asm : `mov rax, [rbx+i*8]` pour accéder au `i`-ème pointeur, puis `mov al, [rax]` pour lire le premier char de la string.

► Solution

Exercice H4 — Struct sur le heap (malloc + struct + pointeurs)

Crée une struct `Process` :

```
struct Process {
    int pid;
    char name[32];
    int malicious; // 0 ou 1
};
```

Écris :

- `struct Process *create_process(int pid, char *name, int mal)` → alloue un `Process` sur le heap, remplit les champs, retourne le pointeur
- `void print_process(struct Process *p)` → affiche les infos
- `void kill_process(struct Process *p)` → affiche un message et `free`

Dans `main` :

- Crée 3 processus avec `create_process`
- Affiche-les
- Free-les avec `kill_process`

Résultat attendu :

```
[PID 1234] svchost.exe      [CLEAN]
[PID 6666] backdoor.exe    [MALICIOUS]
[PID 4321] explorer.exe    [CLEAN]
Killed PID 1234
Killed PID 6666
Killed PID 4321
```

Indice RE : en malware analysis, les structs allouées sur le heap contiennent souvent les config C2 (IP, port, clés). Pattern : `call malloc` → `mov [rax], valeur` → `mov [rax+4], valeur`.

► Solution

Exercice H5 — Use-After-Free (bug classique)

Écris un programme qui **démontre** un use-after-free :

1. `malloc` un buffer de 32 bytes, copie "CONFIDENTIEL" dedans
2. Affiche le contenu
3. `free` le buffer
4. `malloc` un **nouveau** buffer de 32 bytes, copie "AAAAAAAAAA" dedans
5. Affiche l'**ancien** pointeur (celui qui a été free)

Questions à te poser :

- Est-ce que l'ancien pointeur affiche encore "CONFIDENTIEL" ?
- Est-ce qu'il affiche "AAAAAAAAAA" ?
- Pourquoi ?

```
Buffer 1 : CONFIDENTIEL (adresse: 0x5555...)
[free buffer 1]
Buffer 2 : AAAAAAAAAA (adresse: 0x5555...)
Buffer 1 (après free) : AAAAAAAAAA <-- USE AFTER FREE!
```

Indice RE : le UAF est une des vulnérabilités les plus exploitées. Après `free`, l'allocateur peut réutiliser la même zone mémoire. L'ancien pointeur pointe vers les nouvelles données → fuite d'info ou contrôle du flux d'exécution.

► Solution

Exercice H6 — Double Free (crash contrôlé)

Écris un programme qui :

1. `malloc` un buffer
2. `free` le buffer
3. `free` le buffer **une deuxième fois** (double free)

Compile et exécute :

```
gcc -g -O0 -o double_free double_free.c
./double_free
```

Observe le message d'erreur (`double free or corruption`).

Puis corrige le bug : après chaque `free`, mets le pointeur à `NULL` et vérifie avant de `free`.

Indice RE : le double free corrompt les métadonnées internes du heap (les free lists). Les exploits heap utilisent ça pour obtenir un `write-what-where` — écrire une valeur arbitraire à une adresse arbitraire.

- Solution (version buggée)
- Solution (version corrigée)
-

Exercice H7 — Memory leak detector maison (malloc + compteur)

Écris un mini-système de tracking mémoire :

- Variable globale `int alloc_count = 0`
- Fonction `void *tracked_malloc(size_t size)` → appelle `malloc`, incrémente `alloc_count`, affiche l'allocation, retourne le pointeur
- Fonction `void tracked_free(void *ptr)` → appelle `free`, décrémente `alloc_count`, affiche la libération
- Fonction `void check_leaks(void)` → affiche si `alloc_count == 0` (pas de leak) ou combien de blocs fuient

Dans `main`, fais exprès de :

- Allouer 5 blocs
- N'en libérer que 3
- Appeler `check_leaks` → doit détecter 2 leaks

Résultat attendu :

```
[+] malloc 32 bytes -> 0x5555... (total: 1)
[+] malloc 64 bytes -> 0x5555... (total: 2)
[+] malloc 16 bytes -> 0x5555... (total: 3)
[+] malloc 128 bytes -> 0x5555... (total: 4)
[+] malloc 8 bytes -> 0x5555... (total: 5)
[-] free 0x5555... (total: 4)
[-] free 0x5555... (total: 3)
[-] free 0x5555... (total: 2)
[!] LEAK DETECTED: 2 blocs non libérés!
```

Indice RE : les outils comme Valgrind font exactement ça (en plus sophistiqué). En CTF, les leaks heap peuvent être exploités pour lire des adresses et contourner l'ASLR.

► Solution

Exercice H8 — Heap buffer overflow (vulnérabilité)

Écris un programme avec deux buffers **consécutifs** sur le heap :

```
char *username = malloc(16);
char *role = malloc(16);
strcpy(role, "user");
```

Puis lis un username avec `gets` (ou `strcpy` d'une string trop longue) qui **dépasse** les 16 bytes et **écrase** le contenu de `role`.

Objectif : faire en sorte que `role` devienne `"admin"` sans jamais écrire directement dans `role`.

Résultat attendu :

```
role avant : user
Entrez username: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
role après : AAAA... (écrasé!)
```

Note : le comportement exact dépend de l'allocateur. Compile avec `gcc -g -O0 -z execstack -fno-stack-protector` pour maximiser les chances de voir l'overflow.

Indice RE : le heap overflow est la technique derrière beaucoup d'exploits réels. Les métadonnées malloc (chunk headers) entre les blocs peuvent aussi être corrompues.

► Solution

Récap patterns RE (heap)

Pattern C	Ce que tu vois en assembleur
<code>malloc(n)</code>	<code>mov rdi, n → call malloc → résultat dans rax</code>
<code>free(p)</code>	<code>mov rdi, p → call free</code>
<code>calloc(n, s)</code>	<code>mov rdi, n → mov rsi, s → call calloc</code>
<code>realloc(p, n)</code>	<code>mov rdi, p → mov rsi, n → call realloc</code>
Use-After-Free	accès via un pointeur après un <code>call free</code>
Double Free	deux <code>call free</code> avec le même argument
Heap Overflow	écriture au-delà de la taille allouée

Checkpoint

- ☐ H1 : `realloc` — tableau qui grandit dynamiquement
- ☐ H2 : `calloc` vs `malloc` — observer l'initialisation
- ☐ H3 : tableau de strings (`malloc` de pointeurs + `malloc` de buffers)
- ☐ H4 : struct sur le heap (`create/print/kill`)
- ☐ H5 : `use-after-free` — observer la réutilisation mémoire
- ☐ H6 : `double free` — crash + correction
- ☐ H7 : leak detector maison
- ☐ H8 : heap buffer overflow — écraser un buffer adjacent